

DB Health Portal · Non-Exadata Edition

Technical Documentation

Architecture, components, data sources, configuration, deployment and security

Document control	
Product	DB Health Portal (Oracle Non-Exadata Database Monitoring Dashboard)
Build	dashboard_1.9.8 – DB_Health_Portal_new
Document ID	DBHP-TD-001
Version / Date	1.0 · 08-Oct-2026
Intended audience	Developers, DBAs and support engineers who build, deploy, maintain or audit the portal
Prepared by	yuvashanthi

All screenshots in this document were captured from the real application running against **fictitious demo data** (databases PRODFIN, PRODHR, PRODCRM, PRODERP, PRODSCM, PRODBI on hosts 10.20.30.41–43). No production host names, users or credentials are shown.

Contents

1. System overview	9. Configuration files
2. Technology stack	10. Runtime files & logs
3. Architecture	11. Build & deployment
4. Source layout	12. Security considerations
5. Runtime flow	13. Known issues & limitations
6. Module reference	14. Operations & troubleshooting
7. Oracle data sources	15. Appendix A – Screens
8. Health and alert logic	16. Appendix B – DASHBOARD_PKG

1. System overview

The DB Health Portal is a single-user Windows desktop application. It monitors many Oracle databases running on conventional (non-Exadata) Linux or Windows servers. It is a **Streamlit** web application frozen with **PyInstaller** into `DB_Health_Portal.exe`. The exe starts a local web server on port **8501** and opens a browser tab.

Data is collected in two ways:

- **Directly from Oracle:** SQL against `v$` / `DBA_` views over Oracle Net, using the `python-oracledb` driver.
- **From the host OS:** over **SSH** (paramiko), for CPU, RAM, processes and mount points.

A background thread refreshes a status cache every 120 seconds and sends email alerts on state changes.

2. Technology stack

Layer	Component	Version in build
Runtime	CPython (embedded by PyInstaller)	3.10
UI framework	Streamlit	1.44.1
Charts	Plotly	6.7.0
Data	pandas / numpy	numpy 2.2.6
Oracle driver	python-oracledb: thin mode in the main process, thick mode (Instant Client in <code>_internal/oracle_client</code>) for the reporting-DB check subprocess	3.4.1
SSH	paramiko	5.0.0
Crypto	cryptography, bcrypt, PyNaCl (paramiko dependencies)	cryptography 46.0.3
Packaging	PyInstaller (one-dir build, spec <code>_rebuild/DB_Health_Portal.spec</code>)	–

3. Architecture



Key design points

- **Two collection paths.** The UI collects data itself through Streamlit fragments and thread pools (up to 30 parallel database checks on Home, 20 parallel queries on the detail page). Separately, `monitor_thread` builds the shared disk cache that the sidebar and the alert engine use.
- **Reporting DB check in a subprocess.** It runs as a separate process (the exe called with `--check-reporting-db`), so thick-mode initialisation never conflicts with the thin connections in the main process (DPY-2019).
- **Fallbacks.** If a direct Oracle connection fails and host credentials exist, queries can run through `sqlplus` over SSH (`SSHConnection`). OS metrics fall back from SSH to `V$OSSTAT` / `V$SYSMETRIC`.

4. Source layout

Path (under <code>DB_Health_Portal_new/</code>)	Purpose
<code>DB_Health_Portal.exe</code>	PyInstaller launcher (entry point <code>run_dashboard.py</code>).
<code>_internal/app.py</code>	Streamlit entry script: shutdown handling, query-parameter routing, sidebar, Mail History page, page router.
<code>_internal/dashboard/home.py</code>	Registry setup screen, Portal Home cards, KPI tiles, host resources, health categorisation.
<code>_internal/dashboard/monitoring.py</code>	Database detail page: all panels, kill session, archive log clear.
<code>_internal/dashboard/mail_config.py</code>	SMTP settings and notification preferences page.
<code>_internal/dashboard/app.py</code>	Legacy entry script. Not used at runtime.
<code>_internal/db_connection.py</code>	Registry parsing, connections (pool, direct, SID fallback, SSH sqlplus), thick-mode bootstrap, reporting check.
<code>_internal/queries/queries.py</code>	All SQL used by the UI (inline statements).
<code>_internal/queries/dashboard_pkg.sql</code>	PL/SQL package <code>DASHBOARD_PKG</code> (26 ref-cursor procedures). Optional; see Appendix B.

<code>_internal/monitor_thread.py</code>	Background collector, cache writer and alert trigger (active).
<code>_internal/monitor_daemon.py</code>	Older standalone collector loop. Not imported.
<code>_internal/utils/alerts.py</code>	Email alert rules, de-duplication state, SMTP send, history.
<code>_internal/utils/ssh_*_provider.py</code>	SSH collectors for mount points, processes and CPU/RAM.
<code>_internal/utils/storage_provider.py</code>	Platform-specific storage/process provider; optional Java-in-DB bridge.
<code>_internal/utils/storage.py</code>	Developer test script. Not imported; must not be imported (see section 12).
<code>generate_db_status_log.py</code>	Writes <code>db_status_report.log</code> after each monitor cycle.
<code>_internal/assets/style.css</code>	Global stylesheet; dark-mode overrides are injected by <code>app.py</code> .

5. Runtime flow

- Launch:** `run_dashboard.main()` calls `startup_registry_check()`, which discards any pending registry path, then `streamlit run _internal/app.py --global.developmentMode=false`.
- First script run:** `monitor_thread.start_monitor_thread()` starts one daemon thread per process. A new browser session sets `session_config_done=False`, so the Registry Setup screen is shown.
- Registry load:** `save_txt_path()` stores the path in `db_path_config.json` (keeping a history of the last 10) and writes `config/force_refresh.flag` so the monitor runs a cycle immediately.
- Portal Home:** `render_home_dashboard_fragment` claims the databases not yet in `status_cache` and runs `load_db_status_summary_basic` in a `ThreadPoolExecutor` (max 30). Each card is painted as its result arrives. Reporting checks follow (max 5 parallel subprocesses, 15 s timeout each). Host summaries then load per host.
- Detail page:** `render_dashboard(db)` runs 17 collectors in parallel (max 20 threads) and caches the result in session state for 60 s.
- Background cycle** (every 120 s): `collect_db_metrics` runs for up to 8 databases in parallel. After each database, the cache is written and `check_and_trigger_alerts` runs; at the end of the cycle, `generate_report()` runs.
- Shutdown:** the Shutdown button, or 10 s with no browser session, stops the monitor thread, closes the pool and calls `os._exit(0)`.

6. Module reference

6.1 db_connection.py

Function	Description
<code>get_txt_path()</code>	Active registry path: session state first, then the thread-safe global, then <code>db_path_config.json</code> .
<code>save_txt_path</code> / <code>save_pending_path</code> / <code>confirm_pending_path</code> / <code>discard_pending_path</code> / <code>clear_active_registry</code> / <code>get_registry_history</code>	Registry path life-cycle and history (last 10).
<code>read_db_file_to_df(path)</code> , <code>parse_smart_txt_to_df</code>	Parse .txt (whitespace, positional, # comments, - = empty) and .csv/.xlsx (header auto-detected). Cached by file modification time.
<code>load_db_names()</code>	Unique database names (case-insensitive) in file order.
<code>get_config_for_db(db)</code>	Returns {user, password, dsn=host:port/service, host, port, service_name, host_username, host_password}; None for NC lines.
<code>get_reporting_db_config(db)</code>	Same, built from columns 9–14.
<code>get_host_ssh_credentials(host)</code>	First non-empty host credentials on any line for that host; the user defaults to <code>opc</code> .
<code>get_oracle_mode(user)</code>	<code>sys</code> → SYSDBA; <code>sysoper</code> / <code>sysbackup</code> / <code>sysdg</code> / <code>syskm</code> → their privilege modes; otherwise default authentication.
<code>get_api_connection(db)</code>	Direct connect with a 2 s TCP timeout. Retries with an explicit SID= descriptor on DPY-6001, ORA-12505 or ORA-12154.
<code>get_connection_pool()</code> / <code>get_db_connection()</code>	Session pool (min 1, max 5, increment 1) for the selected database, then direct connect, then the SSH sqlplus fallback.
<code>execute_query</code> / <code>execute_query_to_df</code> / <code>execute_proc_to_df</code> / <code>execute_non_query</code>	Query helpers. <code>execute_proc_to_df</code> reads a REF CURSOR OUT parameter.
<code>SSHConnection</code> / <code>SSHCursor</code>	DB-API-like wrapper that runs <code>sqlplus -S</code> with CSV markup on the database host over SSH.
<code>_ensure_thick_mode()</code>	Thread-safe <code>init_oracle_client</code> . Searches <code>_internal/oracle_client</code> , <code>PATH</code> and <code>ORACLE_HOME</code> for <code>oci.dll</code> .
<code>check_reporting_db_status(db)</code>	Spawns <code>exe --check-reporting-db db registry result.json log</code> (15 s timeout) and returns {configured, status, listener, reporting_db_name, error, response_time_ms, checked_at, ...}.

6.2 monitor_thread.py

Constants: `POLL_INTERVAL = 120 s`; `MAX_PARALLEL_DBS = 8`.

Files it uses: `config/db_monitor_cache.json`, `config/daemon_status.json` and `config/force_refresh.flag`.

`collect_db_metrics(db)` gathers, for one database:

- Database and listener status, the latest backup, and permanent-tablespace usage.
- Archive destination use: FRA or custom.
- Blocking sessions: up to 10.
- Deadlocks in the alert log in the last hour.
- ORA errors and OS resources.
- `session_maxed` : true when sessions exceed the larger of 50 or 85% of the `sessions` parameter.
- The reporting database check.

Public functions: `start_monitor_thread` , `stop_monitor_thread` , `trigger_force_refresh` , `read_monitor_cache` , `read_monitor_status` .

6.3 utils providers

Module	Remote commands / behaviour
ssh_mount_provider	<code>df -Pk</code> plus <code>/proc/mounts</code> (falls back to <code>df -Pk</code>); pseudo filesystems and zero-size entries filtered out. Timeouts: connect 6 s, auth 8 s, command 8 s. Alternative <code>get_mounts_via_oracle_scheduler</code> uses a DBMS_SCHEDULER job and UTL_FILE.
ssh_process_provider	<code>ps -eo user,pid,ppid,pcpu,pmem,rss,etime,args</code> filtered to the <code>oracle</code> user, mapped to SIDs through <code>V\$PROCESS</code> . Falls back to <code>V\$SESSION</code> / <code>V\$SESSTAT</code> CPU deltas. Per-host cache TTL 30 s.
ssh_resource_provider	<code>nproc</code> , <code>free -b</code> (uses the <i>available</i> column), <code>vmstat 1 2</code> (falls back to <code>top -bn1</code>).
storage_provider	Picks a Windows, Linux or macOS provider from <code>V\$DATABASE.PLATFORM_NAME</code> . Optional Java stored procedure <code>StorageScanner</code> (deploy_java_bridge). Final fallback: local psutil.
alerts	Alert rules, state de-duplication in <code>alert_state.json</code> , SMTP send, history (last 200) and toast notifications (last 20).

7. Oracle data sources

Function (queries.py)	Views / tables	Used for
get_db_status	V\$INSTANCE	Status, instance, host, version, startup time
get_session_stats	V\$SESSION	Active/inactive user sessions
get_tablespace_utilization	DBA_DATA_FILES, DBA_FREE_SPACE	% used against autoextend maximum, datafile location
get_backup_status / get_rman_durations	V\$RMAN_BACKUP_JOB_DETAILS	Latest backup; 7-day durations
get_session_license	V\$LICENSE	Current sessions and high-water mark
get_db_growth_rates	DASHBOARD_GROWTH_TRACK (custom table)	Growth trends
get_cpu_consuming_sessions	V\$SESSION, V\$SESSTAT, V\$STATNAME, V\$SQL	Top 10 CPU sessions, excluding SYS, SYSTEM, DBSNMP, SYSMAN, SYSDG, SYSBACKUP, SYSKM, SYSRAC
get_blocking_sessions / get_lock_waits	V\$SESSION, V\$SQL, V\$LOCK	Blocker → waiter pairs
get_alert_log	V\$DIAG_ALERT_EXT	ORA- messages, last 2 days, up to 50
get_max_sessions	V\$PARAMETER ('sessions')	Session limit
get_asm_storage	V\$ASM_DISKGROUP, V\$ASM_DISK	ASM disk groups
get_system_resources	V\$OSSTAT, V\$SYSMETRIC, V\$SGA, V\$PROCESS	CPU/RAM when SSH is not available
get_sga_pga_usage	V\$SGA, V\$SGASTAT, V\$PGASTAT	Memory panel
get_listener_status	V\$LISTENER_NETWORK, V\$INSTANCE	Listener state
get_arc_log_info	V\$ARCHIVE_DEST, V\$RECOVERY_FILE_DEST, V\$ARCHIVED_LOG	Archive destination usage

Minimum privileges. The account needs `SELECT_CATALOG_ROLE` (or `SELECT ANY DICTIONARY`). Two features need more:

- Kill Session requires `ALTER SYSTEM`.
- The optional Java bridge and scheduler-based mount collection require `CREATE PROCEDURE`, `CREATE JOB`, `CREATE ANY DIRECTORY` and Java permissions.

Do not grant these unless the feature is needed.

`DASHBOARD_GROWTH_TRACK` is not created by the application. If it is missing or has fewer than 2 rows, the growth panel shows **fixed placeholder values** (2.9 / 20.3 / 90.7 / 1088.3 MB), not measured data.

8. Health and alert logic

8.1 Health category (`home.determine_health_category`)

```
if db in (UNKNOWN, NOT_CONFIGURED)                -> Unknown
elif db == DOWN or listener == DOWN or backup == FAILED or log_gap_alert -> Critical
elif backup in (PENDING, WARNING) or full_ts (permanent TS >= 90%)
    or has_blocking or deadlock_detected or session_maxed or alert_log_error -> Warning
else                                                -> Healthy
```

Backup is SUCCESS or FAILED only when the latest RMAN job started 1 day ago or less; otherwise it is PENDING. Home cards count only status `OPEN` as UP. The detail page also accepts `MOUNTED` and `OPEN_MIGRATE` as UP.

8.2 Email alert rules (`utils/alerts.check_and_trigger_alerts`)

Alert	Condition	Preference key
Database Down	db == DOWN	db_down
Listener Down	listener == DOWN	listener_down
Tablespace Nearly Full	tablespace pct ≥ 90	tablespace_critical
Archive Log Issue	arc_pct ≥ 90, or alert log contains ORA-00257 / archiver error	archive_log_full
Mount Point Nearly Full	pct ≥ 90 (not pseudo)	mount_critical
ORA Error	each new ORA-nnnnn code (last 50 remembered)	ora_error
Deadlock	deadlock_detected	deadlock
Blocking Session	has_blocking	blocking_sessions

Alerts fire only when a database moves into the bad state; the state is keyed by `host:db` and resets on recovery. When `enabled` is false, the alert is only logged, with status *disabled*.

9. Configuration files

File	Keys / content
_internal/db_path_config.json	txt_path , history[] (10), pending_txt_path
config/<registry-name>.json (falls back to email_config.json)	smtp_server, smtp_port, smtp_username, smtp_password, from_email, to_emails[], enabled, preferences{32 booleans}
config/alert_state.json	Per host:db : db_down, listener_down, tablespaces{}, arc_log_alert, mount_points{}, ora_errors_sent[], deadlock_alert, blocking_alert
config/db_monitor_cache.json	{db: stats} written by the monitor thread
config/daemon_status.json	fetching, last_cycle_start, last_cycle_end
config/force_refresh.flag	Presence triggers an immediate monitor cycle
config/email_history.json / email_notifications.json	Alert history (last 200) and toast queue (last 20)
TNS_ADMIN (environment variable)	Oracle Net config directory used in thick mode. The code default is a Linux path, so set it explicitly when you need sqlnet.ora or a wallet.

Example email configuration (demo values):

```
{
  "smtp_server": "smtp.demo-corp.local", "smtp_port": 587,
  "smtp_username": "dba-alerts@demo-corp.local", "smtp_password": "*****",
  "from_email": "dba-alerts@demo-corp.local",
  "to_emails": ["dba-team@demo-corp.local", "oncall@demo-corp.local"],
  "enabled": true,
  "preferences": { "db_down": true, "listener_down": true, "tablespace_critical": true, "...": false }
}
```

10. Runtime files & logs

File	Written by	Content
oracle_thick_mode_debug.log	db_connection	Thick-mode init and reporting check trace
normal_db_debug.log	db_connection	Per-database UP/DOWN trace
db_status_report.log	generate_db_status_log	Status of every database and reporting database after each cycle
alerts_log.txt	alerts	Every alert email (sent, failed or disabled)
scratch/session_history.csv	monitoring	24 h session history for the HWM chart
scratch/rpt_<db>_<id>.json/.log	reporting check	Temporary subprocess result; deleted after it is read
_launch_stdout.log / _launch_stderr.log	launcher	Console output

Remote side effects (only when those features are used): `/tmp/dash_df.txt` , directory object `DASH_TMP_DIR` , scheduler job `DASH_DF_JOB` , Java source `StorageScanner` and functions `get_host_storage_info` / `get_host_process_info` .

11. Build & deployment

11.1 Build

1. Python 3.10 environment with streamlit, plotly, pandas, oracledb, paramiko, psutil, openpyxl and pyinstaller.
2. Copy Oracle Instant Client (19c or later, 64-bit) into `_internal/oracle_client` .
3. Build: `pyinstaller _rebuild/DB_Health_Portal.spec` . The spec:
 - bundles `config/` , `assets/` , `queries/dashboard_pkg.sql` , `oracle_client/` and `app.py` as data;
 - copies the full streamlit, plotly and altair folders;
 - adds the streamlit metadata and the hidden imports for `dashboard.*` , `utils.*` , tornado and watchdog.
4. Output: `dist/DB_Health_Portal_new/` (one-dir build). Zip and distribute it.

Before packaging, remove developer files from `config/` that contain real SMTP or database credentials. Ship only empty templates.

11.2 Deployment

- Unzip to a local folder the user can write to; the app writes config and log files next to itself.
- Network: outbound TCP 1521 (or the listener ports used) to every database and reporting database, TCP 22 to the hosts, and SMTP 587/465 to the mail relay.
- Run as a normal (non-admin) Windows user.
- Bind Streamlit to localhost (see section 12, item 9).

11.3 Running from source (developers)

```
cd DB_Health_Portal_new\_internal
py -3.10 -m streamlit run app.py --server.address 127.0.0.1 --server.port 8501
```

12. Security considerations

Review findings from the source code. Fix these before using the portal in production.

#	Finding	Location	Recommendation
1	SMTP host and account are hard-coded and override the configured settings for any port other than 465.	utils/alerts.py ~293-297	Remove them, use the configured settings, and rotate the exposed password .
2	<code>starttls()</code> is called without an SSL context or timeout.	utils/alerts.py ~295	Pass <code>ssl.create_default_context()</code> and a timeout.
3	The test script contains hard-coded host credentials and opens SSH at import.	utils/storage.py	Delete it from the build.
4	Database, OS, SMTP and reporting passwords are stored in plain text (registry, config JSON, monitor cache) and written to logs or stdout.	db_connection.py ~41, 59, 958; monitor_thread.py; run_dashboard.py	Mask passwords in logs. Keep secrets in Windows Credential Manager or an Oracle wallet. Restrict NTFS permissions on the folder.
5	SSH accepts any host key (<code>AutoAddPolicy</code>).	All SSH providers	Load <code>known_hosts</code> and use <code>RejectPolicy</code> ; prefer key-based authentication.
6	Shell command built from registry values; the password is passed on the sqlplus command line.	db_connection.py ~1408-1418	Quote the values (<code>shlex.quote</code>) and pass the password on stdin.
7	Optional Java bridge grants <code><<ALL FILES>></code> read/execute; the scheduler job runs <code>/bin/bash</code> .	storage_provider.py, ssh_mount_provider.py	Keep these disabled; use SSH instead.
8	Oracle Net traffic is unencrypted TCP.	DSN construction	Enable TCPS or native network encryption.
9	The Streamlit server address is not set, so it may listen on all interfaces.	run_dashboard.py	Add <code>--server.address 127.0.0.1</code> .
10	Kill Session and Archive Log Clear run without confirmation; Mail History "login" reuses the SMTP password.	monitoring.py, app.py	Add confirmation dialogs and a separate role or credential.
11	<code>sys</code> users connect as SYSDBA.	get_oracle_mode	Use a least-privilege monitoring account.

13. Known issues & limitations

- **Notification preferences.** Only 8 of the 32 notification preferences have an alert rule; the rest are stored but unused.
- **Fixed thresholds.** Thresholds are hard-coded and not configurable.
- **Inconsistent mount-point colours.** Home uses >90 / >75 , the detail page uses ≥ 90 / ≥ 70 , and alerts use ≥ 90 .
- **Connection banner.** The Home banner always reads "All database(s) reachable"; the diagnostic function behind it is not called.

- **Growth panel.** Shows placeholder values when `DASHBOARD_GROWTH_TRACK` is missing (section 7).
- **Session HWM history.** A new history file is seeded with 10 simulated points.
- **Status can show UP too early.** `monitor_thread` may mark a database UP when the direct connection fails but SSH login succeeds, before any query returns. `generate_db_status_log` does the same. The deadlock fallback also counts ordinary row-lock waits as deadlocks.
- **CPU bar colours.** The Top CPU Sessions bar colour compares hundredths of a second against thresholds in seconds.
- **Mount alerts.** They never fire from the background loop, because it collects no drive list.
- **Leftover bugs.**
 - `monitor_daemon.py` is not used and contains column-name bugs.
 - `storage_provider.py` imports a non-existent `get_db_names`, so its SSH branch falls back to the local machine.
- **Single-user design.** State is per process. Running two instances on one machine shares the `config/` files.

14. Operations & troubleshooting

Problem	Check
DPY-2019 (thick/thin conflict)	The reporting check must run in its subprocess. Look in <code>oracle_thick_mode_debug.log</code> .
DPY-3010 / old database version	Thin mode needs Oracle 12.1 or later; older databases need thick mode.
Reporting DB always DOWN	Make sure <code>oracle_client/oci.dll</code> is present and the subprocess finishes within 15 s.
No OS metrics	SSH password login must be allowed and port 22 open. The <code>oracle</code> user needs to be able to run <code>ps</code> , <code>df</code> and <code>free</code> .
Stale sidebar colours	Delete <code>config/db_monitor_cache.json</code> , or click Refresh on Home.
Port 8501 in use	Close the other instance; the launcher does not choose another port.

Appendix A – Screens (demo data)

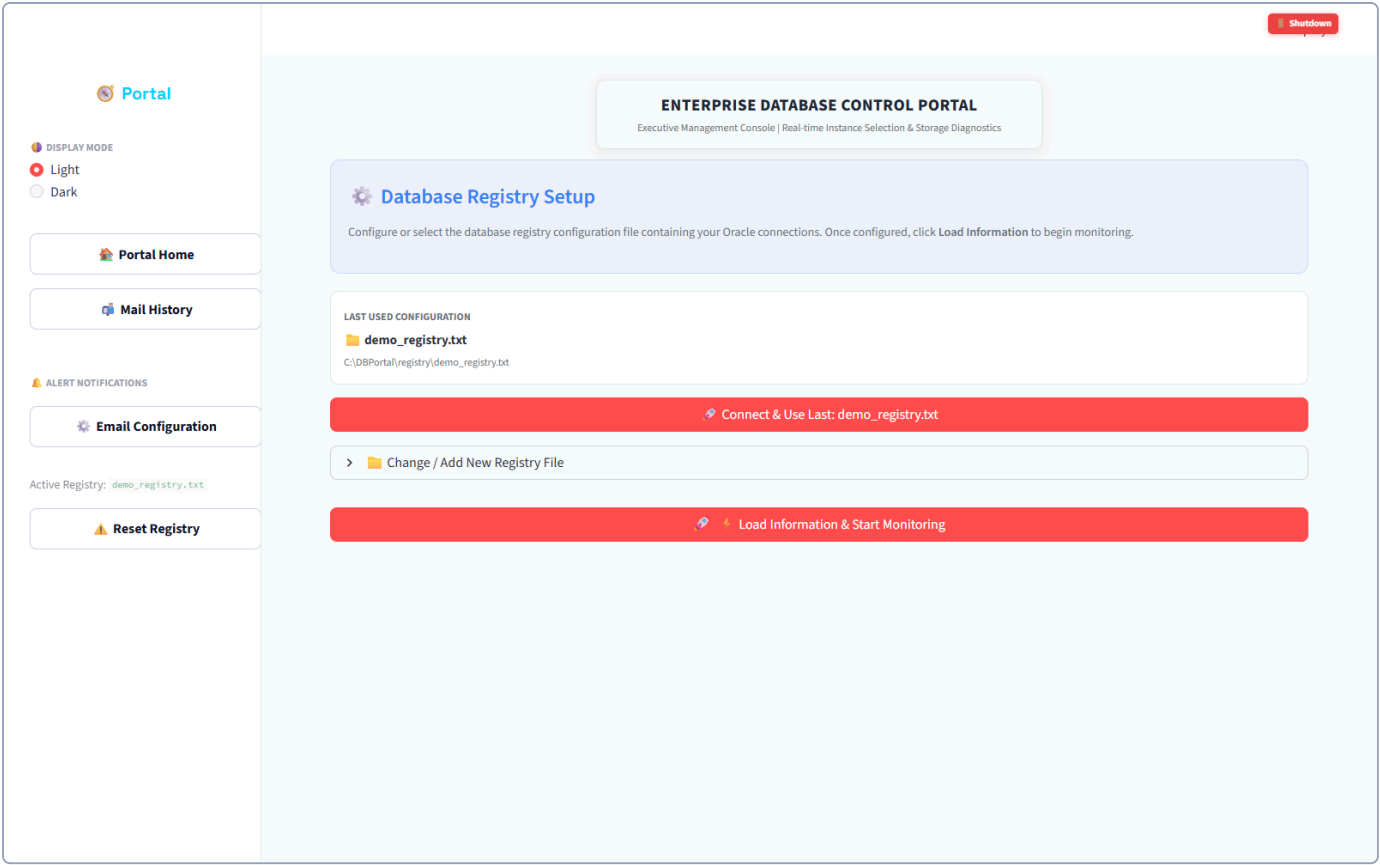


Figure 1: Registry setup (demo data)

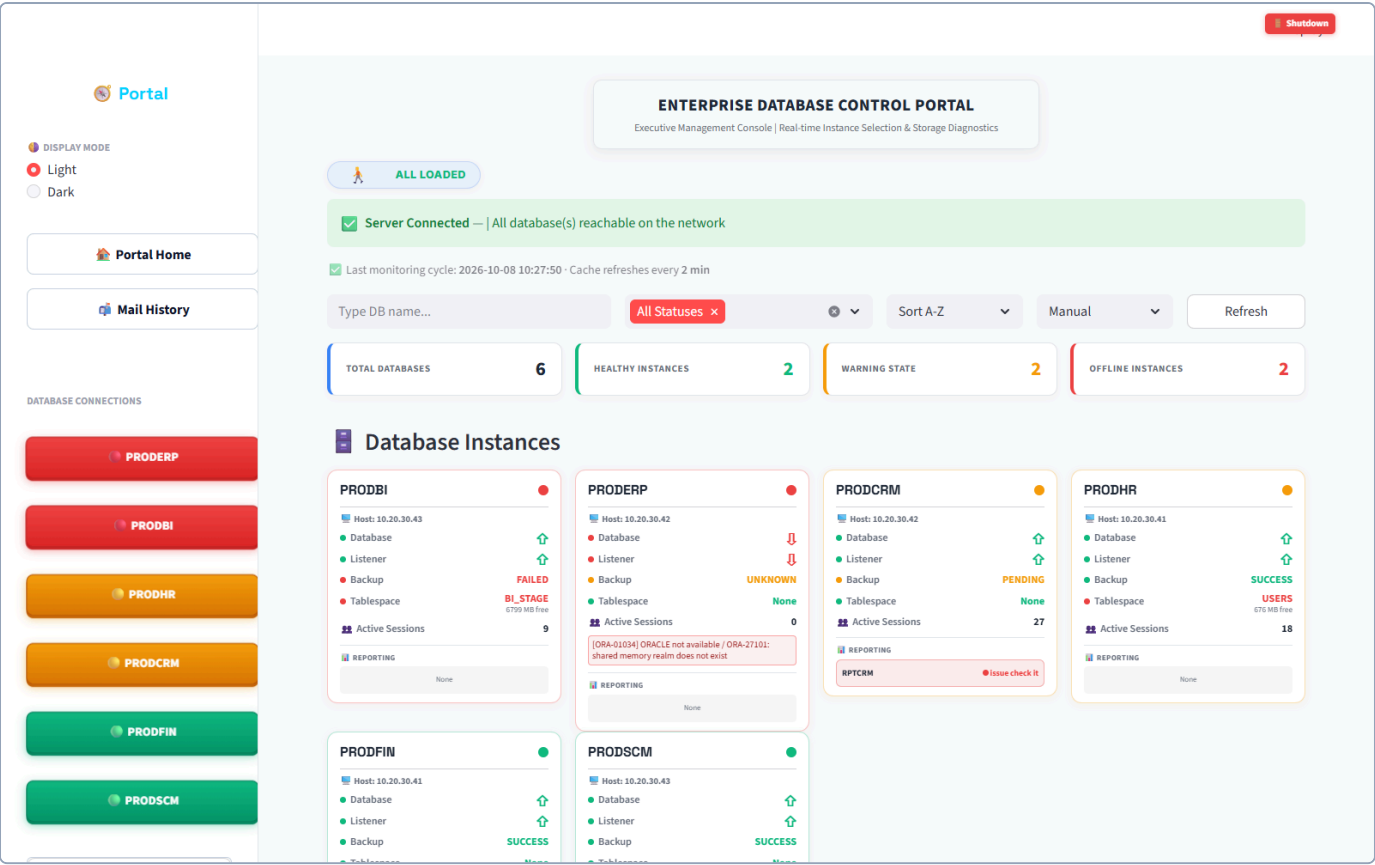


Figure 2: Portal Home (demo data)

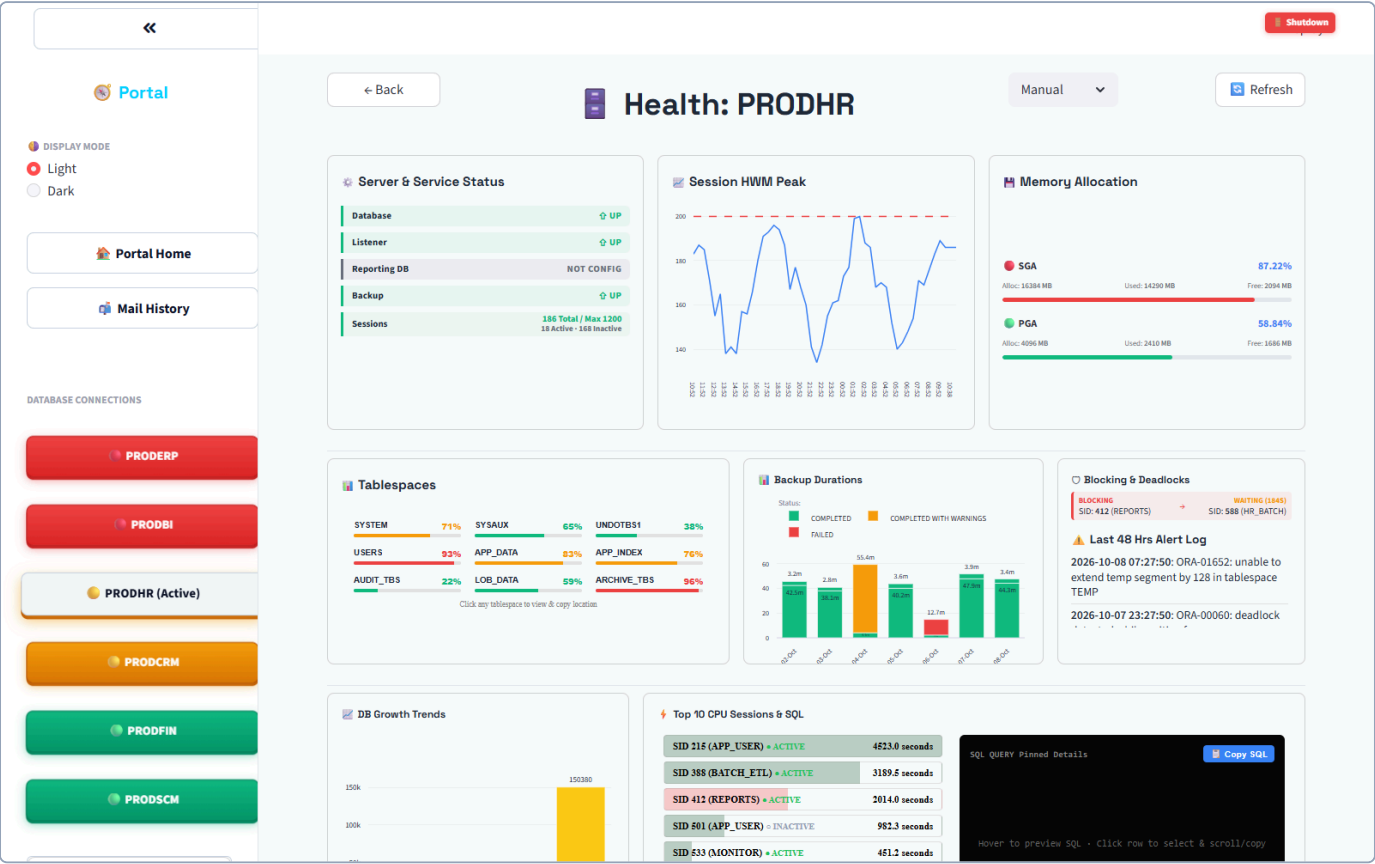


Figure 3: Database detail page (demo data)

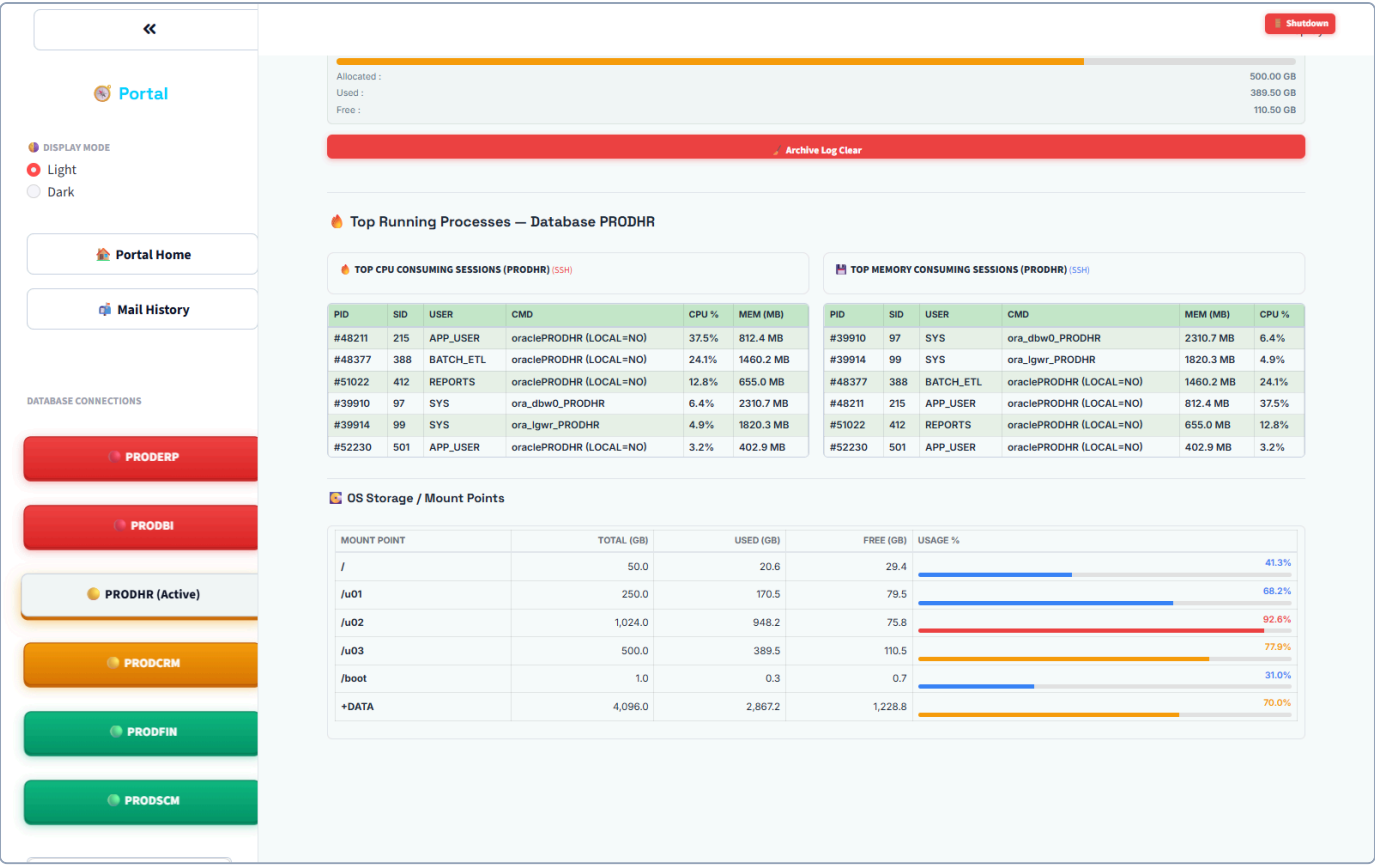


Figure 4: Processes and storage panels (demo data)

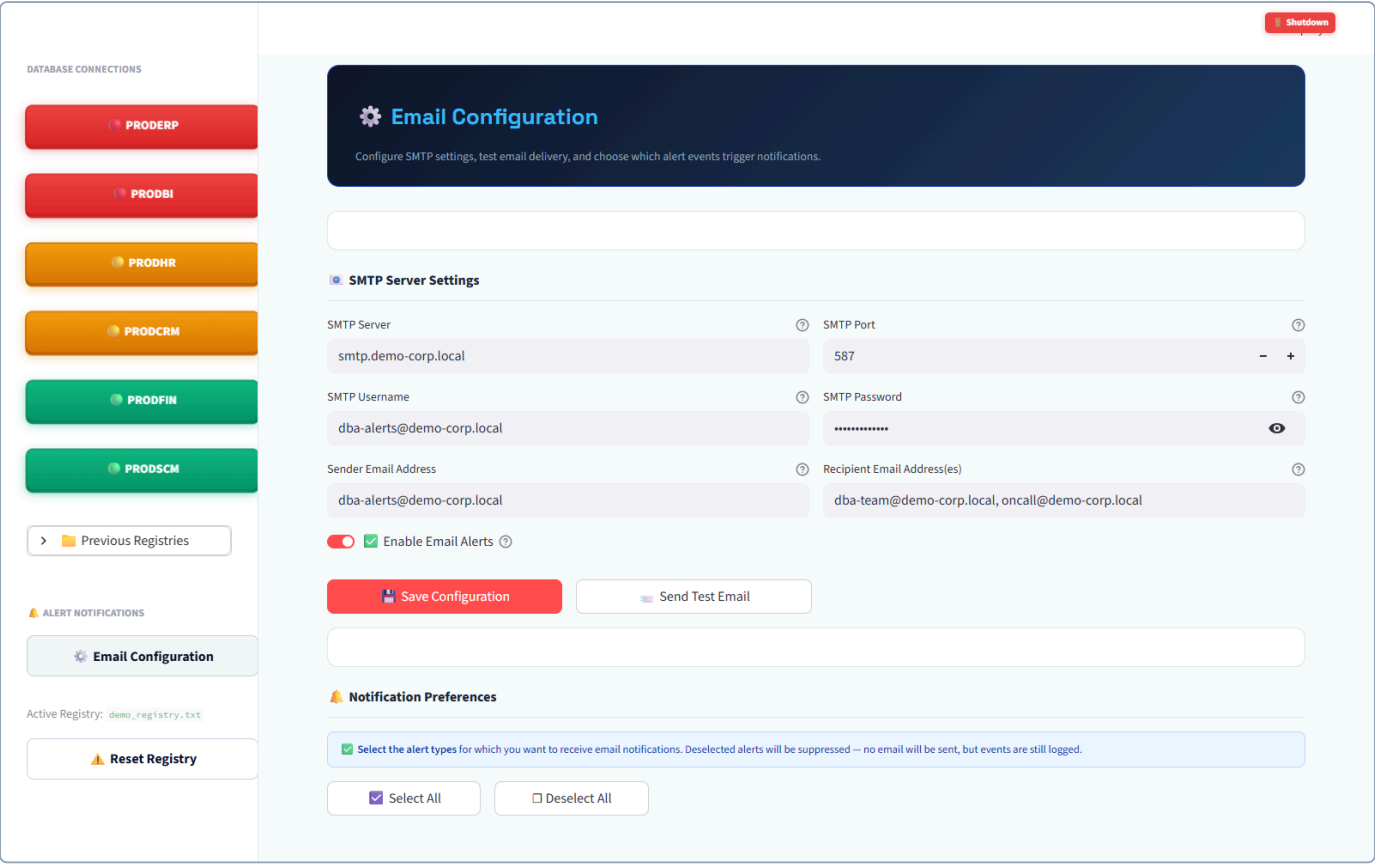


Figure 5: Email configuration (demo data)

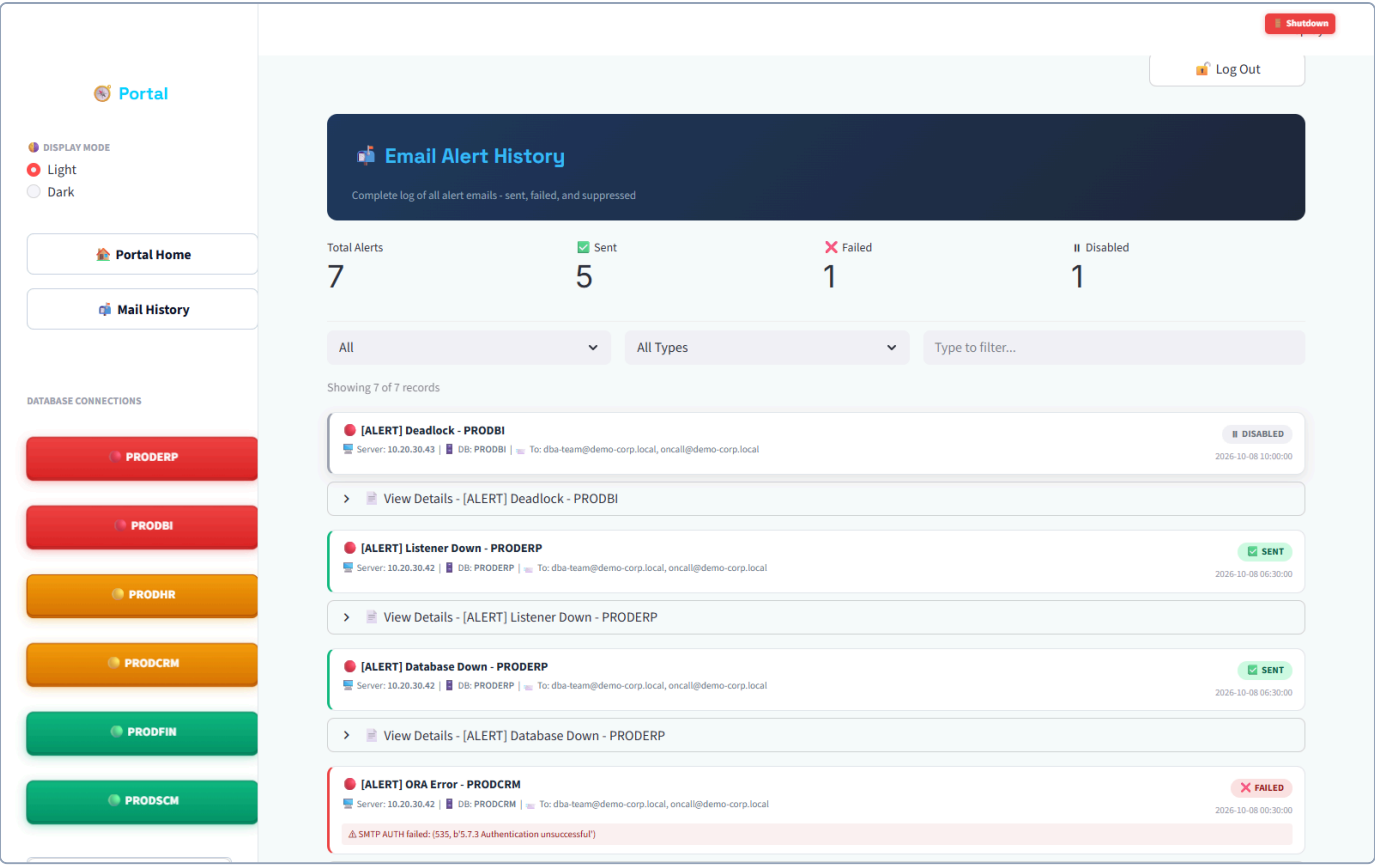


Figure 6: Mail History (demo data)

Appendix B – DASHBOARD_PKG

`queries/dashboard_pkg.sql` creates package `DASHBOARD_PKG`. It has 26 procedures, each with the signature `PROCEDURE get_x(p_recordset OUT SYS_REFCURSOR)`, mirroring the inline SQL. The current UI uses inline SQL; only the legacy daemon calls `get_oracle_mem`. Deploying the package is optional.

Procedure	Columns
<code>get_db_status</code>	<code>status</code> , <code>instance_name</code> , <code>host_name</code> , <code>version</code> , <code>startup_time</code>
<code>get_session_stats</code>	<code>status</code> , <code>cnt</code>
<code>get_tablespaces</code>	<code>tablespace_name</code> , <code>allocated_mb</code> , <code>free_mb</code> , <code>used_mb</code> , <code>maxsize_mb</code> , <code>free_on_max_mb</code> , <code>pct_used_max</code> , <code>location</code>
<code>get_backup_status</code> / <code>get_rman_durations</code>	<code>status</code> / <code>start_time_str</code> , <code>duration_min</code> , <code>status</code> , <code>input_type</code>
<code>get_session_license</code> / <code>get_max_sessions</code>	<code>sessions_current</code> , <code>sessions_highwater</code> / <code>name</code> , <code>value</code>
<code>get_db_growth</code>	<code>record_date_str</code> , <code>used_mb</code> , <code>total_mb</code>
<code>get_cpu_sessions</code>	<code>sid</code> , <code>username</code> , <code>program</code> , <code>cpu_time_val</code> , <code>sql_text</code> , <code>sql_id</code>
<code>get_blocking_sessions</code> / <code>get_lock_waits</code>	<code>blocker/blocked SID</code> , <code>user</code> , <code>program</code> , <code>wait details</code> , <code>SQL</code>
<code>get_alert_log</code>	<code>time_str</code> , <code>message_text</code>
<code>get_asm_storage</code>	<code>disk_group</code> , <code>total_mb</code> , <code>free_mb</code> , <code>state</code> , <code>type</code> , <code>disk_count</code>
<code>get_os_stats</code> / <code>get_sysmetric</code> / <code>get_oracle_mem</code>	OS statistics, system metrics, <code>sga_bytes</code> / <code>pga_bytes</code>
<code>get_arc_log</code> / <code>get_arc_dest</code> / <code>get_arc_fra</code> / <code>get_arc_custom</code>	Archive log destination and usage
<code>get_sga_total</code> / <code>get_sga_free</code> / <code>get_pga_stats</code>	Memory statistics
<code>get_listener_network</code> / <code>get_listener_instance</code>	Listener state
<code>get_pid_sid_map</code>	<code>os_pid</code> , <code>oracle_sid</code> , <code>username</code>